LAB 3
Building RESTful APIs, Routing, Testing with Postman & API Docs with Swagger

Course: Web Application and Database Development

1. Core Theory Summary

1.1 RESTful API Architecture Principles

REST (Representational State Transfer) is the most widely used architectural style for web APIs today. Every **resource** is identified by a URL (a plural noun, e.g. `/api/products`, not `/api/getProducts`) and manipulated with standard HTTP methods. Each request is **stateless**: it carries everything the server needs to process it.

HTTP Method	CRUD Operation	Meaning & Purpose	Typical Status Codes
GET	Read	Retrieve a list or the details of a resource. Does not modify data (safe & idempotent).	200 OK, 404 Not Found
POST	Create	Create a new resource. Data is sent in the request body.	201 Created, 400 Bad Request
PUT	Update (full)	Replace the entire resource identified by an ID.	200 OK, 400 Bad Request, 404 Not Found
PATCH	Update (partial)	Update one or more fields of a resource.	200 OK, 400 Bad Request, 404 Not Found
DELETE	Delete	Remove a resource from the system by ID.	200 OK / 204 No Content, 404 Not Found

Parameter Extraction Techniques in Express.js

Express Syntax	Location in the Request	Purpose & Context	Example
`req.params`	Route path (URL parameters)	Identify a specific resource by ID	`/api/products/:id` → `req.params.id`

Express Syntax	Location in the Request	Purpose & Context	Example
req.query	Query string (after ?)	Filtering, sorting, pagination. Values are always **strings** – convert with Number().	/api/products?page=1&limit=10
req.body	HTTP request body (JSON)	Send complex data on POST, PUT, PATCH (needs express.json())	req.body.name, req.body.price
req.headers	HTTP request headers	Auth info, tokens, Content-Type	req.headers.authorization

1.2 The MVC / Layered Architecture Pattern

To avoid cramming all the logic into a single file ("spaghetti code"), a professional backend project is organized in **layers**. It is often called MVC; in a REST API the "View" is simply the JSON response. Each layer has one clear responsibility:

- **Router layer** (routes/): receives HTTP requests, matches the URL and method, and forwards the request to the right controller.
- **Controller layer** (controllers/): reads the input (req.params, req.query, req.body), calls the service/model, checks the result, and sends the JSON response.
- **Service layer** (services/): the core business logic (calculations, transactions, coordinating several models), independent of Express.
- **Model layer** (models/): defines the schema and constraints and talks to the database (MongoDB via Mongoose, or MySQL via mysql2/Sequelize).

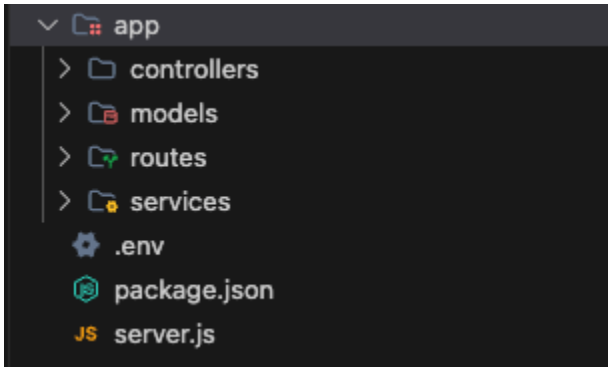


Figure 1.1: Example project structure

1.3 Express Middleware & the Middleware Pipeline

Middleware in Express.js are functions that receive the request (req), the response (res) and a next() function that passes control on. They act as filters in a pipeline between the client's request and the final controller, and they run **in the order they are registered** with app.use().

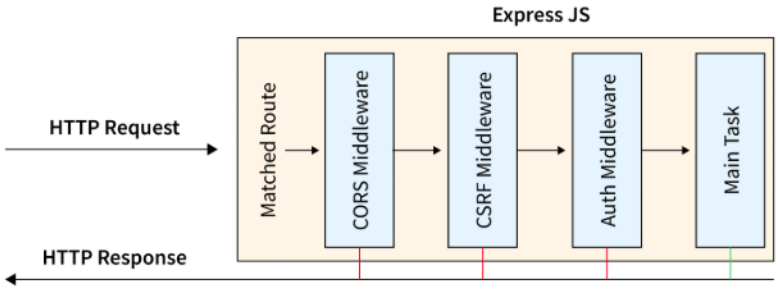


Figure 1.2: The Express middleware pipeline chain

Example: a request-logging middleware

```
// Runs for every request, then hands control to the next handler
const logger = (req, res, next) => {
  console.log(new Date().toISOString(), req.method, req.originalUrl);
  next();
};

app.use(logger);
```

> **Golden rule of middleware: next()**
>
> If a middleware does not end the request-response cycle itself (with res.json() or res.send()), it **must** call next() to pass control on. If you forget, the client's request hangs until it times out.
>
> Calling next(err) with an argument skips the normal middleware and jumps straight to the **error-handling middleware** – the one with four parameters (err, req, res, next).

1.4 Auto-Generating API Documentation with Swagger (OpenAPI 3.0)

OpenAPI 3.0 is the industry-standard specification for describing RESTful APIs; **Swagger** is the family of tools built around it (formerly the specification itself was called "Swagger"). With the swagger-jsdoc and swagger-ui-express libraries, developers write @swagger (or @openapi) JSDoc comments directly in their route files; the comments are scanned into an OpenAPI spec and rendered as an interactive UI at /api-docs.

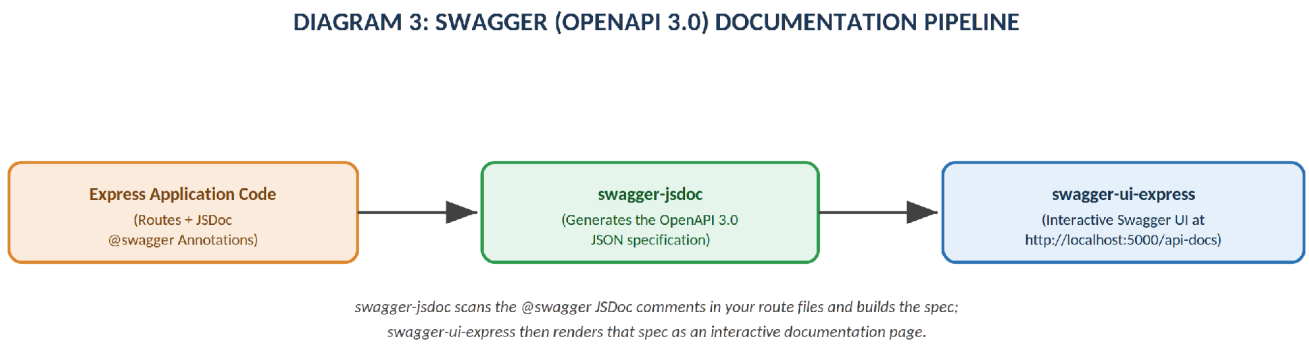


Figure 1.3: The automated JSDoc-scanning and Swagger UI rendering pipeline

2. In-Class Exercises

Exercise 1: A Standard RESTful API with Express Router and Mongoose

Objective: Split the application into modules (Routes, Controllers, Models) and implement a standard RESTful CRUD API for **Products** using Express Router.

Step 0 Create the project

Terminal

```
mkdir lab3-api && cd lab3-api
npm init -y
npm install express mongoose
mkdir models controllers routes config
```

Make sure MongoDB is running locally (mongodb://127.0.0.1:27017) before you start the server.

Step 1 Define the model

models/product.model.js

```
// Mongoose schema definition for Product
const mongoose = require('mongoose');

const productSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Product name cannot be empty'],
    trim: true
  },
  category: {
    type: String,
    required: [true, 'Category is required'],
    enum: ['Electronics', 'Audio', 'Accessories']
  },
```

```
  price: {
    type: Number,
    required: [true, 'Product price is required'],
    min: [0, 'Product price cannot be negative']
  },
  stock: {
    type: Number,
    default: 0
  },
  description: {
    type: String,
    trim: true
  }
}, { timestamps: true });

module.exports = mongoose.model('Product', productSchema);
```

Step 2 Implement the controller

controllers/product.controller.js

```
// Request handlers for the Product API
const Product = require('../models/product.model');

// 1. GET /api/products - List products (filters & pagination)
exports.getAllProducts = async (req, res, next) => {
  try {
    const { category, minPrice, maxPrice } = req.query;
    // Query values are strings: convert them and fall back to safe defaults
    const page = Math.max(Number(req.query.page) || 1, 1);
    const limit = Math.max(Number(req.query.limit) || 5, 1);

    const query = {};
    if (category) query.category = category;
    if (minPrice || maxPrice) {
      query.price = {};
      if (minPrice) query.price.$gte = Number(minPrice);
      if (maxPrice) query.price.$lte = Number(maxPrice);
    }

    const skip = (page - 1) * limit;
    const totalProducts = await Product.countDocuments(query);
    const products = await Product.find(query)
      .sort({ createdAt: -1 })
      .skip(skip)
      .limit(limit);

    res.status(200).json({
      success: true,
      count: products.length,
```

```
      pagination: {
        currentPage: page,
        totalPages: Math.ceil(totalProducts / limit),
        totalProducts
      },
      data: products
    });
  } catch (error) {
    next(error);
  }
};

// 2. GET /api/products/:id - Get product details by ID
exports.getProductById = async (req, res, next) => {
  try {
    const product = await Product.findById(req.params.id);
    if (!product) {
      return res.status(404).json({ success: false, message: 'No product found
with the given ID' });
    }
    res.status(200).json({ success: true, data: product });
  } catch (error) {
    next(error);
  }
};

// 3. POST /api/products - Create a new product
exports.createProduct = async (req, res, next) => {
  try {
    const newProduct = await Product.create(req.body);
    res.status(201).json({ success: true, message: 'Product created
successfully', data: newProduct });
  } catch (error) {
    next(error);
  }
};

// 4. PUT /api/products/:id - Update a product by ID
// Note: findByIdAndUpdate only changes the fields that are sent (PATCH-like).
// A strict PUT would replace the whole document (e.g. findOneAndReplace).
exports.updateProduct = async (req, res, next) => {
  try {
    const updatedProduct = await Product.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true }
    );
    if (!updatedProduct) {
```

```
      return res.status(404).json({ success: false, message: 'No product found
to update' });
    }
    res.status(200).json({ success: true, message: 'Product updated
successfully', data: updatedProduct });
  } catch (error) {
    next(error);
  }
};

// 5. DELETE /api/products/:id - Delete a product by ID
exports.deleteProduct = async (req, res, next) => {
  try {
    const deletedProduct = await Product.findByIdAndDelete(req.params.id);
    if (!deletedProduct) {
      return res.status(404).json({ success: false, message: 'No product found
to delete' });
    }
    res.status(200).json({ success: true, message: 'Product deleted
successfully' });
  } catch (error) {
    next(error);
  }
};
```

Step 3 Configure the router

routes/product.routes.js

```
// Express Router configuration
const express = require('express');
const router = express.Router();
const productController = require('../controllers/product.controller');

router.get('/', productController.getAllProducts);
router.post('/', productController.createProduct);
router.get('/:id', productController.getProductById);
router.put('/:id', productController.updateProduct);
router.delete('/:id', productController.deleteProduct);

module.exports = router;
```

Step 4 Set up the server

server.js

```
// Main server entry point
const express = require('express');
const mongoose = require('mongoose');
const productRoutes = require('./routes/product.routes');
```

```
const app = express();
const PORT = 3000;

app.use(express.json());

// Register API routes
app.use('/api/products', productRoutes);

// Global error handler (must be registered AFTER the routes)
app.use((err, req, res, next) => {
  console.error('Server Error:', err.message);

  let status = err.status || 500;
  if (err.name === 'ValidationError') status = 400; // schema validation failed
  if (err.name === 'CastError') status = 400;       // malformed ObjectId
  if (err.code === 11000) status = 409;             // duplicate unique key

  res.status(status).json({
    success: false,
    error: err.message || 'Internal server error'
  });
});

mongoose.connect('mongodb://127.0.0.1:27017/shop_lab3_db')
  .then(() => {
    app.listen(PORT, () => console.log('Server API is running at:
http://localhost:' + PORT));
  })
  .catch((err) => console.error('MongoDB connection failed:', err.message));
```

Why map error types in the error handler?

Without it, a request with a missing `name` or a negative `price` would return **500 Internal Server Error**, although the problem is the client's input. Mongoose throws a `ValidationError` (→ 400), a malformed ID throws a `CastError` (→ 400), and a duplicate value on a `unique` field throws error code `11000` (→ 409 Conflict).

Verify in the browser

Run `node server.js`, add a product to the `products` collection (use a category from the enum, e.g. "Electronics"), then open **http://localhost:3000/api/products** in your browser (= `GET /api/products`).

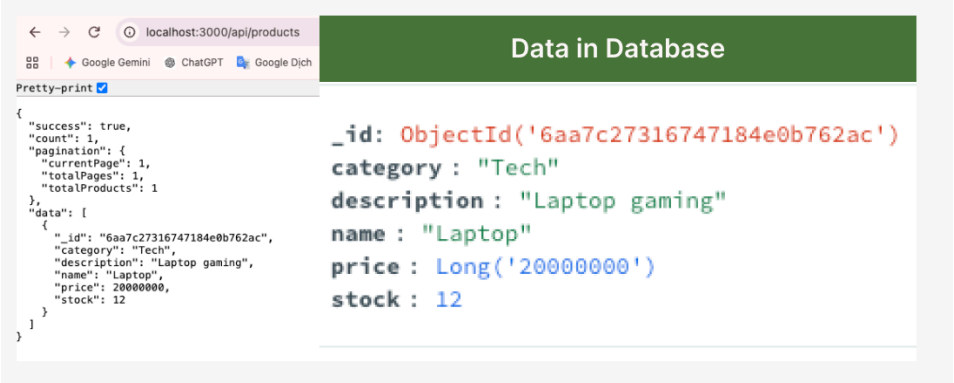


Figure 2.1: GET /api/products in the browser and the stored document

Verify with Postman

Send POST http://localhost:3000/api/products with a raw JSON body. Expect **201 Created** and the new product in data. Then try an invalid body (no name, negative price) and check that you get **400**.

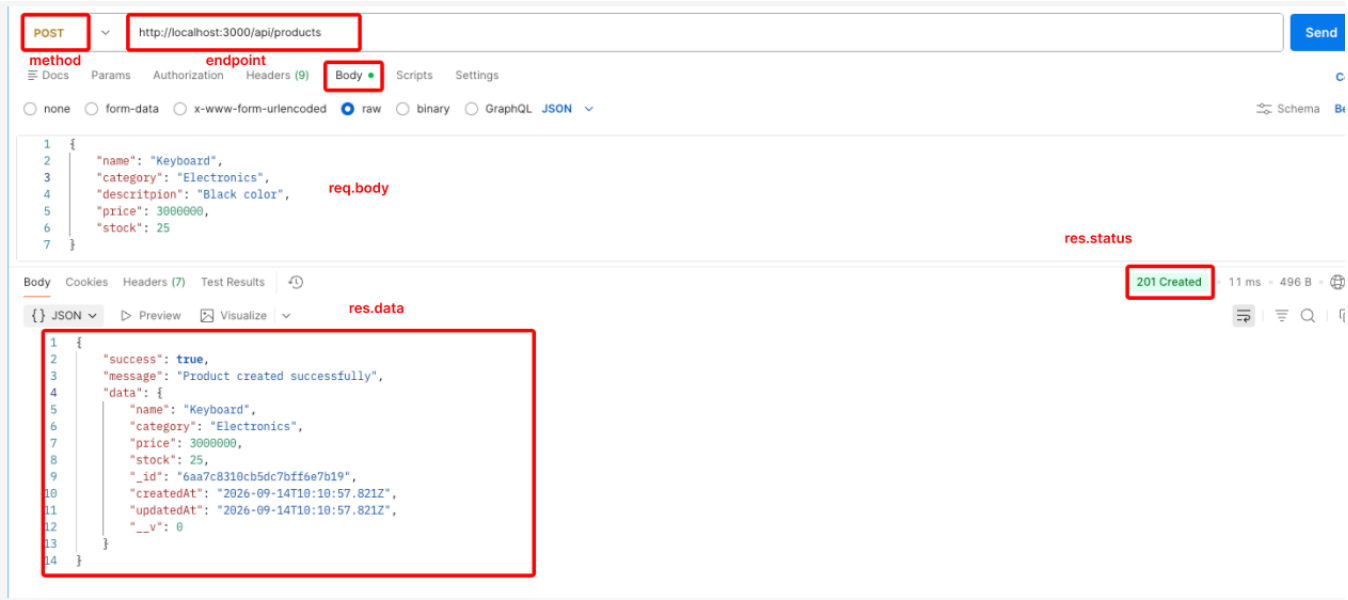


Figure 2.2: Creating a product with Postman

Extended requirement: a full Router & Controller module for Category (/api/categories)

1. Implement full CRUD for **Category** with the fields name, slug, description (MongoDB adds _id automatically).
2. Add an input-validation middleware: if name is empty or shorter than 3 characters, immediately return **400 Bad Request** with an error message.
3. Auto-generate slug from name (e.g. "Home & Living" → "home-living") in a Mongoose pre('save') hook. **Hint:** save hooks do not run on findByIdAndUpdate, so regenerate the slug yourself when name changes in PATCH.

4. Make `name` and `slug` **unique** and return **409 Conflict** with a friendly message when a duplicate is submitted (error code `11000`).
5. `GET`, `PATCH` and `DELETE /api/categories/:id` must return **404** for a valid ID that does not exist and **400** for a malformed ID.
6. Add `PATCH /api/categories/:id` for partial updates (e.g. only `description`), using `runValidators: true`.
7. Add pagination and sorting to `GET /api/categories`, following the same `page`/`limit` pattern as the Products endpoint.

How it evolves in production (Route → Controller → Service → Model)

In real-world applications, business logic moves out of controllers into a dedicated **service layer**:

1. **Separation of concerns:** controllers act purely as HTTP adapters (read `req`, send `res`), while services execute the business rules (calculations, payment integrations, database transactions).
2. **Reusability & testability:** logic that does not depend on `req`/`res` can be reused from WebSockets, background workers (cron, RabbitMQ) or CLI scripts, and is much easier to unit-test.

Exercise 2: Professional API Documentation with Swagger (OpenAPI 3.0)

Objective: Integrate Swagger UI into the Express application with `swagger-jsdoc` and `swagger-ui-express` to produce an interactive OpenAPI 3.0 documentation page.

Step 1 Install the libraries

Terminal

```
npm install swagger-ui-express swagger-jsdoc
```

Step 2 Swagger configuration

config/swagger.js

```
// Swagger / OpenAPI 3.0 configuration
const swaggerJSDoc = require('swagger-jsdoc');
const swaggerUi = require('swagger-ui-express');

const options = {
  definition: {
    openapi: '3.0.0',
    info: {
      title: 'RESTful API Documentation - Web & Database Lab 3',
```

```
      version: '1.0.0',
      description: 'Interactive API documentation auto-generated by Swagger UI
for the Sales Management application'
    },
    servers: [
      {
        url: 'http://localhost:3000/api',
        description: 'Development Local Server'
      }
    ]
  },
  // Files to scan for @swagger comments (relative to the folder you run "node
server.js" from)
  apis: ['./routes/*.js', './models/*.js']
};

const swaggerSpec = swaggerJSDoc(options);

const setupSwagger = (app) => {
  app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerSpec));
  console.log('Swagger UI documentation is ready at:
http://localhost:3000/api-docs');
};

module.exports = setupSwagger;
```

Step 3 Swagger annotations on the router

YAML inside the comment is **indentation-sensitive**: keep the spaces exactly as shown.

routes/product.routes.js (add above the routes)

```
/**
 * @swagger
 * tags:
 *   - name: Products
 *     description: Product management endpoints
 *
 * components:
 *   schemas:
 *     Product:
 *       type: object
 *       required:
 *         - name
 *         - category
 *         - price
 *       properties:
 *         _id:
 *           type: string
 *           description: Auto-generated ID from MongoDB
```

```
 *         name:
 *           type: string
 *           example: "Laptop Dell XPS 15"
 *         category:
 *           type: string
 *           enum: [Electronics, Audio, Accessories]
 *           example: "Electronics"
 *         price:
 *           type: number
 *           example: 35000000
 *         stock:
 *           type: integer
 *           example: 10
 */

/**
 * @swagger
 * /products:
 *   get:
 *     summary: Get the list of products
 *     tags: [Products]
 *     parameters:
 *       - in: query
 *         name: category
 *         schema:
 *           type: string
 *         description: Filter by category
 *       - in: query
 *         name: page
 *         schema:
 *           type: integer
 *         description: Current page (default = 1)
 *     responses:
 *       200:
 *         description: Products retrieved successfully
 */
```

Step 4 Mount Swagger UI in the server

server.js (after app.use(express.json()), before the error handler)

```
const setupSwagger = require('./config/swagger');

setupSwagger(app);
```

Verify

Restart the server and open **http://localhost:3000/api-docs** in a web browser.

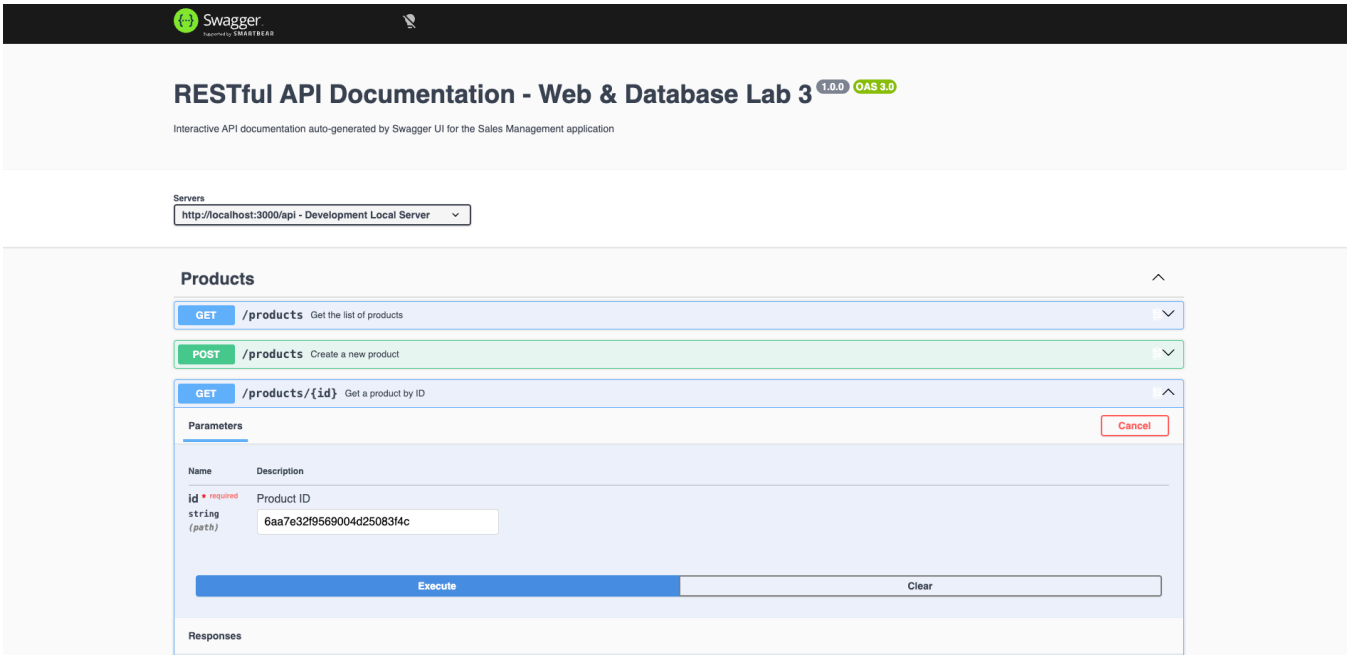


Figure 2.3: The generated Swagger UI

Extended requirement: complete and try out the documentation

1. Add full `@swagger` annotations for `POST /products`, `GET /products/{id}`, `PUT /products/{id}` and `DELETE /products/{id}`, including the **404** response for each `{id}` endpoint.
2. Add full `@swagger` annotations for the **Category** endpoints.
3. Group the endpoints with `tags` so the sidebar is organized (all Product endpoints under **Products**, all Category endpoints under **Categories**).
4. Define a reusable `id` path parameter under `components.parameters` and reference it with `$ref` instead of repeating it on every endpoint.
5. Use **Try it out** to test the POST, GET, PUT and DELETE endpoints directly from the browser.
6. Take a screenshot of Swagger UI showing the Product schema and a successful response, and include it in your report.

3. Homework

Advanced Orders & Inventory Management API with MySQL

Task: Build a production-grade e-commerce orders API with **Node.js, Express.js and MySQL** (`mysql2/promise`), organized in routes / controllers / services as in Exercise 1.

1. Database schema

Create a database named `ecommerce_order_db` with 4 related tables and foreign-key constraints:

Table	Columns
`customers`	`id` INT PK AUTO_INCREMENT · `full_name` VARCHAR(100) · `email` VARCHAR(100) UNIQUE · `phone` VARCHAR(15) · `created_at` DATETIME DEFAULT CURRENT_TIMESTAMP
`products`	`id` INT PK AUTO_INCREMENT · `product_name` VARCHAR(150) · `price` DECIMAL(12,2) · `stock` INT · `created_at` DATETIME DEFAULT CURRENT_TIMESTAMP
`orders`	`id` INT PK AUTO_INCREMENT · `customer_id` INT FK → customers.id · `status` ENUM('pending','completed','cancelled') DEFAULT 'pending' · `total_amount` DECIMAL(12,2) · `created_at` DATETIME DEFAULT CURRENT_TIMESTAMP
`order_items`	`id` INT PK AUTO_INCREMENT · `order_id` INT FK → orders.id **ON DELETE CASCADE** · `product_id` INT FK → products.id · `quantity` INT · `unit_price` DECIMAL(12,2)

Insert **3–5 sample records** into each table.

2. API endpoints

POST **/api/v1/orders** – Create an order (SQL transaction)

- Payload: `{ "customerId": 1, "items": [{ "productId": 1, "quantity": 1 }, { "productId": 2, "quantity": 2 }] }`
- Validate the payload: `items` must be a non-empty array and every `quantity` a positive integer → otherwise **400**.
- Inside one atomic transaction (`beginTransaction`, `commit`, `rollback`):
  - Check that `customerId` exists → otherwise **404 Not Found**.
  - Check that every product exists (→ **404**) and has enough stock (`stock >= quantity`) → otherwise **400** ("Product X is out of stock").
  - Calculate `total_amount` from the **current database prices**.
  - Insert into `orders` and `order_items`.
  - Deduct the stock: `stock = stock - quantity`.
- Return **201 Created** with the new order.

GET **/api/v1/orders** – List orders with filtering, pagination & JOIN

- Query params: `?status=pending&customerId=1&page=1&limit=5`

- JOIN `customers` (and aggregate `order_items`) to return: `order_id`, `customer_name`, `email`, `status`, `total_amount`, `total_items_count` (= SUM of quantities), `created_at`, plus pagination info like Exercise 1.

GET **/api/v1/orders/:id** – Order details

- Multi-table JOIN returning nested JSON: customer info and an array of items (`product_name`, `quantity`, `unit_price`, `subtotal`). **404** if the order does not exist.

PATCH **/api/v1/orders/:id/status** – Update status & restore stock

- Payload: `{ "status": "cancelled" }`. A value outside the ENUM → **400**.
- When the status changes from 'pending'/'completed' to 'cancelled', restore the stock of every item in the **same transaction**. An order that is already cancelled cannot change status again → **400**.

DELETE **/api/v1/orders/:id** – Delete an order

- Delete the order; its `order_items` are removed automatically by `ON DELETE CASCADE`. If the order is not cancelled yet, restore its stock first (same transaction). **404** if it does not exist.

GET **/api/v1/orders/analytics/sales-summary** – Sales revenue analytics

- Use SQL aggregation (SUM, COUNT, AVG, GROUP BY) to return:
  - `totalRevenue`: total amount of 'completed' orders.
  - `completedOrdersCount`: number of completed orders.
  - `averageOrderValue`: average amount per completed order.
  - `topSellingProducts`: the 3 products with the highest total quantity sold in completed orders.

GET **/api/v1/customers/:id/order-history** – Customer order history

- All orders of the customer plus `totalLifetimeSpent` across their completed orders. **404** if the customer does not exist.

Hint – a transaction with mysql2/promise

```
const conn = await pool.getConnection();
try {
  await conn.beginTransaction();
  // SELECT ... FOR UPDATE locks the product rows until commit,
  // so two orders cannot buy the same last item at the same time
  const [rows] = await conn.query(
    'SELECT id, price, stock FROM products WHERE id IN (?) FOR UPDATE',
[productIds]);
  // ... check stock, INSERT orders / order_items, UPDATE products ...
  await conn.commit();
} catch (err) {
```

```
  await conn.rollback();
  throw err;
} finally {
  conn.release();
}
```

3. Testing and documentation

- Screenshots of the test results in **Postman** for every endpoint (success and error cases).
- **Swagger** documentation for all endpoints, tested with Try it out.